

Milestone 3: Reflections

Yucai Zhong and Christina McCallum and Jennifer Flake and Rachelle De Jager

1 Reflection Journal

Please start a new section called Reflection in your Overleaf document. Write a few paragraphs to answer the following questions. Try to be concise. Each team member needs to write their own reflection. But each reflection should not exceed half a page.

1. What do you expect to gain from this shared task and what did you actually gain?
2. Does your task go well? What goes well and what doesn't go well? What is the biggest barrier? Do you feel that you are on the right track?
3. If you could re-start this task again, what are the good practices you want to keep and what are the not-so-good practices you want to improve?

1.1 Yucai Zhong's Reflection

1.2 Christina McCallum's Reflection

1.3 Jennifer Flake's Reflection

1.4 Rachelle De Jager's Reflection

2 Milestone 3 Model

2.1 Model description

For this milestone's model, we decided to combine the useful, effective parts of previous models together. In addition to combining previous model approaches, we added new features based on HT-MIA. By reading "What Hard Tokens Reveal: Exploiting Low-confidence Tokens for Membership Inference Attacks against Large Language Models" by Md Tasnim Jawad, Mingyan Xiao, and Yanzhao Wu, we learned about HT-MIA, which is an approach to MIA that compares token-level probability improvements by extracting features that capture the difference between the probabilities assigned by the fine-tuned model and the probabilities assigned

by the pre-trained reference model on the hardest tokens. As the paper says, "The key intuition is that non-member samples behave similarly for both target (F) and reference (B) models, while the fine-tuned model F assigns systematically higher probabilities than the reference model B to member samples, especially at hard-to-predict tokens with low confidence."

The new HP-MIA features differentiate this model from all of the previous models our group has created. The HT-MIA features we added include $ht_{mia_score_k10}$, $ht_{mia_score_k20}$, and $ht_{mia_score_k30}$. These scores are calculated as $K\%$ of the hardest tokens (ranked by the target model's probability) where $K = 10, K = 20$, and $K = 30$.

The remaining features of the model have been used in previous models. They are: perplexity, loss, $mean_{token_prob}$, min_{token_prob} , max_{token_prob} , std_{token_prob} , $entropy_{token_prob}$.

Continuing the practice of training multiple classifiers and then selecting the best one, we implemented a logistic regression, random forest, gradient boosting and XGBoost. The "results" section of this report contains the performance details of each classifier.

2.2 Implementation

In our implementation for this milestone, we minimized our use of Jupyter Notebooks. Our `model5.ipynb` file serves as an interface for Colab and is mostly used to call functions that exist in our accompanying `.py` script files. In addition, the `model5.ipynb` notebook performs basic functions like mounting the google drive and showing helpful runtime output. Aside from the notebook file, the source for this model includes the following files: `batches.py`, `build_membership_classifier.py`, `compute_perplexity_and_features.py`, `models.py` and `config.py`.

When `model5.ipynb` executes, it runs the following pipeline:

1. Setup. The notebook mounts Google Drive as a save directory and sets the compute device to GPU if available.

2. Data loading. `load_data()` fetches the train, validation, and test splits from HuggingFace.

3. Model loading. `load_models_and_tokenizer()` loads four models from HuggingFace: the fine-tuned target models (SmolLM2-135M and SmolLM2-360M) and their corresponding pre-trained reference models, along with a shared tokenizer.

4. Feature extraction. `process_all_splits()` runs feature extraction across all three splits and all four models. For each text, `compute_perplexity_and_features()` computes perplexity, token-level probability statistics, Min-K% scores, and HT-MIA scores. The reference models are passed in specifically to support the HT-MIA computation. The resulting features are saved to Google Drive via `save_features()` and can be reloaded in a later session with `load_features()` to skip re-extraction.

5. Classifier training. `build_membership_classifier()` combines the 135M and 360M feature sets, adds cross-model delta features, and trains four classifiers: logistic regression, random forest, XGBoost, and gradient boosting. Each is evaluated on the validation set by AUC and TPR at FPR=0.1. The best-performing classifier is selected and saved to Google Drive.

6. Submission generation. `generate_submission()` applies the best classifier to the test set and writes the membership predictions to a CSV file, which is then downloaded from Colab.

2.2.1 Here is a summary of the functions in the code base:

Function: `compute_perplexity_and_features()`

File: `compute_perplexity_and_features.py`

Purpose: Computes perplexity, token-level probability statistics, Min-K% scores, and optional HT-MIA scores for a single input text.

Parameters: `model`, `tokenizer`, `text`, `max_length=512`, `ref_model=None`

Returns: dict of scalar features

Called by: `analyze_sample_batch()`

Function: `analyze_sample_batch()`

File: `batches.py`

Purpose: Extracts features for a batch of texts using a single model.

Parameters: `model`, `tokenizer`, `texts`, `labels`, `model_name`, `ref_model=None`

Returns: feature DataFrame

Called by: `process_dataset_in_batches()`

Function: `process_dataset_in_batches()`

File: `batches.py`

Purpose: Runs feature extraction over a full dataset in batches.

Parameters: `model`, `tokenizer`, `texts`, `labels`, `model_name`, `batch_size=1000`, `ref_model=None`

Returns: aggregated DataFrame

Called by: `process_all_splits()`

Function: `process_all_splits()`

File: `batches.py`

Purpose: Coordinates feature extraction across all train/val/test splits for both models.

Parameters: `model_135m`, `model_360m`, `ref_135m`, `ref_360m`, `tokenizer`, `splits`, `batch_size=500`

Returns: dict of 6 DataFrames

Called by: `model5.ipynb`

Function: `save_features()`

File: `batches.py`

Purpose: Saves extracted feature DataFrames to disk.

Parameters: `save_dir`, `features`

Returns: None

Called by: `model5.ipynb`

Function: `load_features()`

File: `batches.py`

Purpose: Loads previously saved feature DataFrames from disk.

Parameters: `save_dir`

Returns: dict of DataFrames

Called by: `model5.ipynb`

Function: `_build_feature_matrix()`

File: `build_membership_classifier.py`

Purpose: Combines 135m and 360m feature sets and computes cross-model delta features.

Parameters: `df_135m`, `df_360m`

Returns: combined feature DataFrame

Called by: `build_membership_classifier()`, `prepare_test_predictions()`

Function: `compute_zlib_baseline()`

File: `build_membership_classifier.py`

Purpose: Computes zlib compression ratio as a baseline membership inference signal.

Parameters: `val_df`

Returns: dict (auc, tpr)

Called by: model5.ipynb

Function: build_membership_classifier()

File: build_membership_classifier.py

Purpose: Trains and evaluates membership inference classifiers on the combined feature set.

Parameters: train_135m, train_360m, val_135m, val_360m

Returns: dict of classifier results

Called by: model5.ipynb

Function: prepare_test_predictions()

File: build_membership_classifier.py

Purpose: Generates membership probability scores for the test set using a trained classifier.

Parameters: test_135m, test_360m, best_classifier

Returns: np.ndarray

Called by: generate_submission()

Function: generate_submission()

File: build_membership_classifier.py

Purpose: Produces the final submission CSV from test set predictions.

Parameters: test_135m, test_360m, best_classifier, test_df, output_path

Returns: DataFrame (id, is_member)

Called by: model5.ipynb

Function: get_model_dtype()

File: models.py

Purpose: Selects the optimal floating point dtype (bf16 or fp32) based on the device.

Parameters: device

Returns: torch.dtype

Called by: load_models_and_tokenizer()

Function: load_models_and_tokenizer()

File: models.py

Purpose: Loads the fine-tuned target models, reference models, and shared tokenizer.

Parameters: device

Returns: tuple (tokenizer, model_135m, model_360m, ref_model_135m, ref_model_360m)

Called by: model5.ipynb

Function: load_data()

File: models.py

Purpose: Loads the train, validation, and test splits from HuggingFace.

Parameters: —

Returns: dict of DataFrames

Called by: model5.ipynb

Method	AUC	TPR@FPR=0.1
Loss	0.9303	0.7728
Zlib Entropy	0.5075	0.1028

Method	AUC	TPR@FPR=0.1
Loss	0.9279	0.7662
Zlib Entropy	0.5075	0.1028

2.3 Results

2.3.1 XGBoost:

2.3.2 Gradient Boosting:

2.3.3 Random Forest:

2.3.4 Logistic Regression:

Results

All four classifiers operate on the same 55-feature set, which combines per-model base features (perplexity, loss, token probability statistics, entropy) from both the 135M and 360M models, cross-model delta and ratio features, Min-K% scores, and HT-MIA scores. The feature matrix therefore captures both the absolute behaviour of each model and how the two models differ in their confidence on a given text.

Logistic Regression fits a linear decision boundary over the 55 features. It is the simplest of the four classifiers and assumes that membership can be determined by a weighted sum of the input features. It achieved an AUC of 0.9173 and a TPR@FPR=0.1 of 0.7618. The competitive performance of logistic regression suggests that the feature space has strong linear structure and that the engineered features (particularly ratios and deltas) already capture much of the signal without requiring nonlinear interactions. Its main limitation is that it cannot model interactions between features, which likely accounts for the modest gap relative to the tree-based methods.

Random Forest builds an ensemble of decision trees, each trained on a random subset of features and data, and averages their predictions. It achieved an AUC of 0.9228 and the highest TPR@FPR=0.1 of 0.7730. Unlike logistic regression, it naturally captures nonlinear interactions. Its feature importance rankings show that ht_mia_score_k30_360m and

Method	AUC	TPR@FPR=0.1
Loss	0.9228	0.7730
Zlib Entropy	0.5075	0.1028

Method	AUC	TPR@FPR=0.1
Loss	0.9173	0.7618
Zlib Entropy	0.5075	0.1028

`loss_ratio` are the two dominant predictors, with HT-MIA scores from the 360M model occupying three of the top four positions. Random forests can overfit on high-dimensional data and are generally less efficient than boosting methods, which may explain why it did not perform quite as well as XGBoost on AUC despite its strong TPR.

XGBoost uses gradient boosting with regularization to build an ensemble of trees sequentially, each correcting the residual errors of the previous. It achieved the best AUC of 0.9303. Its feature importance is notably concentrated: `ht_mia_score_k30_360m` alone accounts for nearly 25% of the total importance, and the top three features (`ht_mia_score_k30_360m`, `ht_mia_score_k20_360m`, `loss_ratio`) together account for nearly 50%. This concentration suggests XGBoost is efficiently exploiting a small number of highly predictive features.

Gradient Boosting is similar to XGBoost but does not have the same regularization and parallelization optimizations. It achieved an AUC of 0.9279. Its feature importance is even more concentrated than XGBoost: `ht_mia_score_k30_360m`, alone, accounts for nearly 49% and `loss_ratio` for another 27%. The remaining features contribute very little. This extreme concentration may indicate that gradient boosting is over-relying on a small subset of features and not using the rest of the feature set as much as possible.

Error Analysis

Across all four classifiers, TPR@FPR=0.1 sits between 0.76 and 0.77, which means that when the false positive rate is held at 10%, approximately one in four true members is missed. The relatively small spread in TPR@FPR=0.1 across methods (less than 0.02) compared to the larger spread in AUC suggests that the classifiers differ more in their overall ability to classify members vs non-members, than they do at the 10% false positive data point.

The consistent dominance of 360M model features over 135M features across all tree-based methods stands out. HT-MIA scores computed from the 360M model take the top positions in every im-

portance ranking, while 135M HT-MIA scores appear lower in the importance ranking. This suggests that the larger model produces a cleaner memorization signal, which is what we expected because the larger capacity likely supports larger memorization.

The ordering $k_{30} > k_{20} > k_{10}$ in HT-MIA feature importance is consistent across all methods, indicating that examining a larger fraction of hard tokens provides a more reliable signal than focusing only on the very hardest tokens. .

Potential Improvements

Hyperparameter tuning. None of the classifiers were tuned beyond their defaults. Systematic tuning of XGBoost (learning rate, tree depth, subsampling) would likely yield further AUC gains.

Ensembling. The four classifiers have meaningfully different characteristics — logistic regression has the lowest AUC but different error patterns from the tree methods. A soft-voting ensemble over all four could improve both AUC and TPR.

HT-MIA K values. Only K=10%, 20%, and 30% were used. Experimenting with additional values (e.g., K=5%, 40%, 50%) or treating K as a tunable hyperparameter could yield a stronger signal.

Feature selection. Given that gradient boosting assigns near-zero importance to the majority of the 55 features, aggressive feature selection or dimensionality reduction could reduce noise and potentially improve generalization, particularly for logistic regression.

2.4 Kaggle results / observations

Our Kaggle score for the competition this week is 0.93657, which is an improvement over last week's score of 0.90549.

3 Bibliography

Md Tasnim Jawad, Mingyan Xiao, and Yanzhao Wu. 2026. What Hard Tokens Reveal: Exploiting Low-Confidence Tokens for Membership Inference Attacks against Large Language Models. *arXiv preprint arXiv:2601.20885*.

4 Contributions

4.1 Milestone 3

Christina: Jennifer: Implemented and documented model 5. Submitted test results to Kaggle. Wrote personal reflection journal.

Rachelle: Yucai: